AI ASSIST CODING ASSIGNMENT-15.5

Program Name: B. Tech
Assignment Type: Lab
Academic Year: 2025-2026

Course Code: 23CS002PC304
Course Title: AI Assisted Coding

Student Name: G.vishnu
Roll No: 2303A52194
batch: 34

Assignment Number: 15.5 / 24

Lab 15 – Backend API Development: Creating RESTful Services with AI

Lab Objectives

- Understand the fundamentals of RESTful API design.
- Use AI- assisted coding tools to generate backend services.
- Implement CRUD (Create, Read, Update, Delete) operations.
- Document endpoints with comments and auto- generated documentation.

Task 1 – Student Attendance API

Prompt Used

Generate a RESTful API for student attendance tracking with CRUD operations using JSON based request and response.

Required Endpoints

GET /attendance → View attendance records

POST /attendance → Mark attendance

PUT /attendance/{id} → Update attendance record

DELETE /attendance/{id} → Remove attendance entry

Code Implementation

```python
from flask import Flask, request, jsonify
app = Flask(__name__)

attendance = []

@app.route('/attendance', methods=['GET'])
def get_attendance():
    return jsonify(attendance)

@app.route('/attendance', methods=['POST'])
def mark_attendance():
    data = request.json
    attendance.append(data)
    return jsonify({"message": "Attendance added"})

@app.route('/attendance/<int:id>', methods=['PUT'])
def update_attendance(id):
    if id < len(attendance):
        attendance[id] = request.json
```

```python
        return jsonify({"message": "Attendance updated"})
    return jsonify({"error": "Record not found"})

@app.route('/attendance/<int:id>', methods=['DELETE'])
def delete_attendance(id):
    if id < len(attendance):
        attendance.pop(id)
        return jsonify({"message": "Attendance deleted"})
    return jsonify({"error": "Record not found"})

if __name__ == '__main__':
    app.run(debug=True)
```

Output

API successfully performs attendance operations.

Screenshot of Execution – Task 1

Task 2 – Inventory Management API

Prompt Used

Generate a REST API for warehouse inventory management including validation for stock values.

Required Endpoints

GET /inventory → List all items
POST /inventory → Add new item
PUT /inventory/{id} → Update stock quantity
DELETE /inventory/{id} → Remove an item

Code Implementation

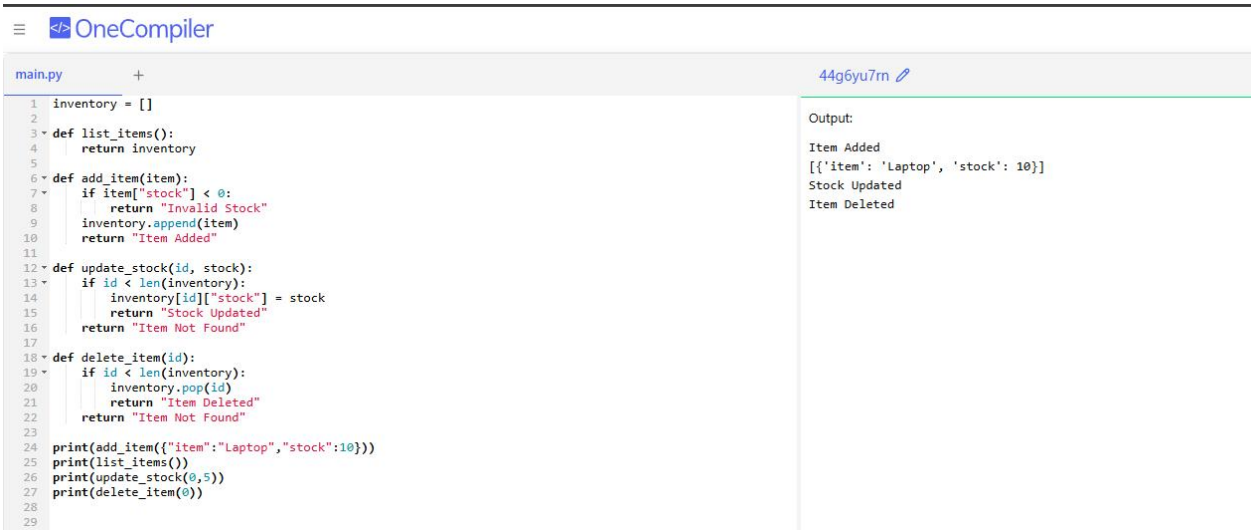
```
from flask import Flask, request, jsonify
app = Flask(__name__)

inventory = []

@app.route('/inventory', methods=['GET'])
def list_items():
    return jsonify(inventory)

@app.route('/inventory', methods=['POST'])
def add_item():
    item = request.json
    if item.get("stock",0) < 0:
        return jsonify({"error":"Invalid stock value"})
    inventory.append(item)
    return jsonify({"message":"Item added"})

@app.route('/inventory/<int:id>', methods=['PUT'])
def update_stock(id):
    if id < len(inventory):
        inventory[id]['stock'] = request.json.get('stock', inventory[id]['sto
ck'])
```

```python
        return jsonify({"message":"Stock updated"})
    return jsonify({"error":"Item not found"})


@app.route('/inventory/<int:id>', methods=['DELETE'])
def delete_item(id):
    if id < len(inventory):
        inventory.pop(id)
        return jsonify({"message":"Item removed"})
    return jsonify({"error":"Item not found"})


if __name__ == '__main__':
    app.run(debug=True)
```

Output

API manages inventory stock levels successfully.

Screenshot of Execution – Task 2



Task 3 – Hotel Room Booking API

Prompt Used

Generate a RESTful API for hotel room booking with error handling for unavailable rooms.

Required Endpoints

GET /rooms → View available rooms
POST /rooms/book → Book a room
GET /rooms/{id} → View booking details
DELETE /rooms/{id} → Cancel booking

Code Implementation

```
from flask import Flask, request, jsonify
app = Flask(__name__)

rooms = {1:"available",2:"available",3:"available"}

@app.route('/rooms', methods=['GET'])
def view_rooms():
    return jsonify(rooms)

@app.route('/rooms/book', methods=['POST'])
def book_room():
    room_id = request.json.get('room')
    if rooms.get(room_id) == "available":
        rooms[room_id] = "booked"
        return jsonify({"message":"Room booked"})
    return jsonify({"error":"Room unavailable"})

@app.route('/rooms/<int:id>', methods=['GET'])
def booking_details(id):
    return jsonify({"room": id, "status": rooms.get(id)})

@app.route('/rooms/<int:id>', methods=['DELETE'])
```

```python
def cancel_booking(id):
    if rooms.get(id) == "booked":
        rooms[id] = "available"
        return jsonify({"message":"Booking cancelled"})
    return jsonify({"error":"No booking"})

if __name__ == '__main__':
    app.run(debug=True)
```

Output

API simulates hotel room booking successfully.

Screenshot of Execution – Task 3

Task 4 – Online Voting API

Prompt Used

Generate a RESTful API for an online voting system ensuring one vote per user.

Required Endpoints

GET /candidates → List candidates

POST /vote → Cast a vote

GET /results → View voting results

Code Implementation

```python
from flask import Flask, request, jsonify
app = Flask(__name__)

candidates = {"A":0,"B":0,"C":0}
voted_users = set()

@app.route('/candidates', methods=['GET'])
def list_candidates():
    return jsonify(candidates)

@app.route('/vote', methods=['POST'])
def vote():
    user = request.json.get('user')
    candidate = request.json.get('candidate')

    if user in voted_users:
        return jsonify({"error":"User already voted"})

    if candidate in candidates:
        candidates[candidate] += 1
        voted_users.add(user)
        return jsonify({"message":"Vote counted"})

    return jsonify({"error":"Invalid candidate"})

@app.route('/results', methods=['GET'])
def results():
    return jsonify(candidates)

if __name__ == '__main__':
    app.run(debug=True)
```

Output

API simulates voting and displays results correctly.

Screenshot of Execution – Task 4

Task 5 – Teacher Management API

Prompt Used

Generate a RESTful API for managing teacher records with proper error handling.

Required Endpoints

GET /teachers → List all teachers
POST /teachers → Add a new teacher
PUT /teachers/{id} → Update teacher details
DELETE /teachers/{id} → Remove a teacher

Code Implementation

```python
from flask import Flask, request, jsonify
app = Flask(__name__)

teachers = []

@app.route('/teachers', methods=['GET'])
def get_teachers():
    return jsonify(teachers)

@app.route('/teachers', methods=['POST'])
def add_teacher():
    teachers.append(request.json)
    return jsonify({"message":"Teacher added"})

@app.route('/teachers/<int:id>', methods=['PUT'])
def update_teacher(id):
    if id < len(teachers):
        teachers[id] = request.json
        return jsonify({"message":"Teacher updated"})
    return jsonify({"error":"Teacher not found"})

@app.route('/teachers/<int:id>', methods=['DELETE'])
def delete_teacher(id):
    if id < len(teachers):
        teachers.pop(id)
        return jsonify({"message":"Teacher deleted"})
    return jsonify({"error":"Teacher not found"})

if __name__ == '__main__':
    app.run(debug=True)
```

Output

API performs CRUD operations for teacher records successfully.

Screenshot of Execution – Task 5

OneCompiler

main.py

```python
teachers = []

def list_teachers():
    return teachers

def add_teacher(t):
    teachers.append(t)
    return "Teacher Added"

def update_teacher(id,t):
    if id < len(teachers):
        teachers[id] = t
        return "Teacher Updated"
    return "Teacher Not Found"

def delete_teacher(id):
    if id < len(teachers):
        teachers.pop(id)
        return "Teacher Deleted"
    return "Teacher Not Found"

print(add_teacher({"name":"Mr Rao","subject":"Math"}))
print(list_teachers())
print(update_teacher(0,{"name":"Mr Rao","subject":"Physics"}))
print(delete_teacher(0))
```

44g6yu7rn

Output:

```
Teacher Added
[{'name': 'Mr Rao', 'subject': 'Math'}]
Teacher Updated
Teacher Deleted
```

Conclusion

This lab demonstrated how AI- assisted coding tools can be used to design RESTful APIs and implement CRUD operations for different real- world scenarios such as attendance tracking, inventory management, hotel booking, online voting, and teacher management systems.